**PROJECT NAME: REPLATE**

**GROUP NUMBER and MEMBERS: GROUP 7: Aylin DÖNMEZ, Beyza Makbule YILDIRIM, Eren PAMUK, Hayrünisa TOPRAK, Hüseyin Alp ERCANDANOĞLU, Hüseyin ESEN, Ramazan YILDIRIM**

## Questions to identify measurements:

**1- How comprehensive and reliable is the automated testing process for the core system functionalities?**

**2- How fast and accurate is the simulated food reservation and handover workflow during User Acceptance Testing (UAT)?**

**3- How stable and responsive is the system under concurrent operations and external API integrations?**

**4- How efficient is the team's agile development process and what is the resulting code quality?**

## Identified measurements:

**1. Measurement: Automated Test Pass Rate**

- **Definition: This measurement evaluates the reliability of the codebase by tracking the percentage of automated Unit and Integration tests that pass successfully.**
- **Method: The Spring Boot test runner (JUnit) will automatically execute all test cases during the build process and calculate the ratio of passed tests to total tests.**

**2. Measurement: End-to-End (E2E) Workflow Success Rate**

- **Definition: This metric measures the stability of the core business logic by tracking the success rate of the entire user journey (the complete workflow from login to final handover) to ensure all subsystems work together seamlessly.**
- **Method: The QA team will divide the number of successfully completed test scenarios by the total number of simulated test attempts recorded in the PostgreSQL test database.**

**3. Measurement: Simulated Food Access Cycle Time**

- **Definition: This measurement monitors the average time elapsed between a tester making a reservation and successfully verifying the QR code during User Acceptance Testing (UAT).**

- **Method: The Spring Boot backend will calculate the time difference between the "reservation confirmed" timestamp and the "QR code verified" timestamp for each simulated test order.**

**4. Measurement: QR Code Verification Success Rate**

- **Definition: This measurement evaluates the reliability of the QR code scanning module as an isolated subsystem, ensuring this specific core feature functions accurately during integration testing and UAT.**
- **Method: The Spring Boot backend will automatically calculate the ratio of successful, verified scans to the total number of simulated scan attempts generated by test scripts in the database.**

**5. Measurement: Concurrent Reservation Conflict Rate**

- **Definition: This measurement monitors the frequency of database errors occurring when multiple simulated virtual users attempt to reserve the same food listing simultaneously during load testing.**
- **Method: The Spring Boot server logs will be analyzed weekly to count the number of failed transactions and PostgreSQL rollbacks caused by concurrent booking attempts generated by stress testing tools.**

**6. Measurement: API Response Latency**

- **Definition: This metric measures the average response time of the Google Maps API to ensure external integration does not degrade system performance.**
- **Method: Spring Boot Actuator (APM) will automatically measure the millisecond delay for every external Google Maps API request, and these metrics will be stored in Prometheus and visualized using Grafana.**

**7. Measurement: Sprint Velocity Achievement**

- **Definition: This metric measures the team's process efficiency by comparing the planned Story Points against the actually completed Story Points per Sprint.**
- **Method: The team will divide the completed Story Points by the initially committed Story Points mapped in GitHub Projects at the end of each Sprint review.**

**8. Measurement: Defect Density**

- **Definition: This measurement tracks the critical bug density encountered during the development process to evaluate software quality.**
- **Method: The team will divide the total number of bugs logged in GitHub Issues by the current KLOC at the end of each Sprint.**

**Measurement storage and collection:**

**1. Measurement: Automated Test Pass Rate**

- **What: The percentage of successfully passed JUnit and Mockito tests.**
- **When: Automatically calculated upon every new code commit or build.**
- **Format: Percentage.**
- **How: Captured and logged automatically by the Spring Boot testing framework and stored in the test execution reports.**

**2. Measurement: End-to-End (E2E) Workflow Success Rate**

- **What: The ratio of successful E2E test workflows to total attempted workflows.**
- **When: Evaluated at the end of each Sprint's QA testing phase.**
- **Format: Percentage.**
- **How: The PostgreSQL database logs every simulated test attempt and its final state, allowing the QA team to calculate the overall success ratio.**

**3. Measurement: Simulated Food Access Cycle Time**

- **What: The time elapsed between a tester confirming a reservation and successfully verifying the QR code.**
- **When: Calculated per transaction during User Acceptance Testing (UAT) sessions.**
- **Format: Time duration (Seconds/Minutes).**
- **How: The Spring Boot backend calculates the difference between the "reservation timestamp" and the "QR verification timestamp" for each specific test order ID, logging it in the PostgreSQL test database.**

**4. Measurement: QR Code Verification Success Rate**

- **What: The ratio of successful QR scans to total simulated scan attempts.**
- **When: Logged continuously during the QA testing phase.**
- **Format: Percentage.**

- **How: The PostgreSQL database automatically logs every scan attempt (success or failure) triggered by the backend test scripts and calculates the overall success ratio.**

**5. Measurement: Concurrent Reservation Conflict Rate**

- **What: The number of transaction failures caused by simulated virtual users trying to book the same food simultaneously.**
- **When: Continuously monitored and reviewed at the end of each week.**
- **Format: Integer count logs.**
- **How: The Spring Boot application logs concurrent request failures and PostgreSQL database rollbacks whenever an overbooking (Optimistic Locking) attempt occurs, which are then analyzed weekly.**

**6. Measurement: API Response Latency**

- **What: The average response time of the Google Maps API integrations.**
- **When: Continuously logged during application usage.**
- **Format: Milliseconds (ms).**
- **How: Spring Boot Actuator automatically monitors the duration of every external Google Maps API request, which are then scraped and stored by Prometheus, and monitored through Grafana dashboards.**

**7. Measurement: Sprint Velocity Achievement**

- **What: The ratio of completed Story Points to the initially planned Story Points.**
- **When: At the end of every Sprint (during the Sprint Review).**
- **Format: Percentage.**
- **How: Recorded by the Scrum Master or development team in GitHub Projects to track and reflect the ratio of completed tasks and Story Points.**

**8. Measurement: Defect Density**

- **What: The number of confirmed bugs per thousand lines of code (KLOC).**
- **When: Evaluated at the end of every Sprint.**
- **Format: Decimal or ratio data.**
- **How: Reported and tracked using GitHub Issues by the development team, then calculated by dividing the total bug count by the current KLOC.**

| Measurement Type | Description | Example Measurements |
|---|---|---|
| **Automated Test Pass Rate** | Evaluates the reliability of the codebase by tracking the percentage of automated tests that pass successfully during the build process. | **98.5% pass rate (450/457 tests)** |
| **End-to-End (E2E) Workflow Success Rate** | Measures the stability of the core business logic by tracking successful simulated whole user journeys to ensure all integrated parts function together seamlessly. | **92% success rate** |
| **Simulated Food Access Cycle Time** | Monitors the average duration between a tester's initial reservation and the final successful collection during User Acceptance Testing (UAT). | **45 seconds (Simulated)** |
| **QR Code Verification Success Rate** | Calculates the percentage of successfully verified QR scans against the total simulated scan attempts to evaluate the reliability of this feature as an isolated subsystem before deployment. | **94.5% success rate** |
| **Concurrent Reservation Conflict Rate** | Monitors the frequency of database transaction failures or rollbacks that occur when multiple virtual users attempt to book the exact same food listing simultaneously during load tests. | **12 conflicts / 1,000 reservations (1.2%)** |
| **API Response Latency** | Measures the average delay in milliseconds for Google Maps API requests to ensure external service dependencies do not degrade overall system performance. | **245 ms average latency** |
| **Sprint Velocity Achievement** | Measures the team's process efficiency by comparing the total number of Story Points actually completed against the Story Points initially planned for the Sprint. | **85% (34/40 Story Points)** |
| **Defect Density** | Tracks the critical bug density encountered during the development process to evaluate software quality and overall system stability. | **3.2 bugs / KLOC** |